CSS234 WEB PROGRAMMING II

LECTURE 02 JSON API & GRPC

Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

Bikeshedding Problem

The Concept

- Coined by C. Northcote Parkinson in 1957.

The Scenario

- A committee is tasked with two items:
  - Approving a **$10 Billion Nuclear Power Plant**.
  - Choosing the **color of the staff bike shed**.

The Outcome

- The Nuclear Plant is approved in minutes because it is so complex that most people don't feel qualified to discuss it.
- The Bike Shed color sparks a 2-hour heated debate because everyone understands paint and wants to contribute their opinion.

Why It Happens

Low Barrier to Entry

- Trivial issues are easy to grasp.
- Anyone can have an opinion on a font, a color, or a naming convention.

The Need to Contribute

- Team members want to feel "helpful."
- If they don't understand the complex core architecture, they will over-compensate by scrutinizing the small things they *do* understand.

Sense of Control

- Mastering a small detail provides a quick "win" and a sense of authority, even if that detail has zero impact on the project's success.

Common Signs in Software Development

- Spending hours debating
  - Tabs vs. Spaces
  - CamelCase vs. snake_case
- Endless discussions on the exact shade of blue for a button while the backend logic is still broken.
- 50+ comments on a Pull Request regarding variable names, but zero comments on a massive security flaw in the logic.

The Negative Impact

Decision Fatigue

- By the time the team reaches critical technical decisions, they are mentally exhausted from trivial debates.

Project Delays

- Valuable engineering hours are wasted on things that could be solved by a "flip of a coin."

Lowered Morale

- High-performers often get frustrated when their complex work is stalled by minor, subjective critiques.

How to Avoid Bikeshedding

Adopt Standards & Specs

- This is why **JSON:API** exists! It provides a "shared convention”, so teams don't have to waste time debating API structures.

Assign a "Decider”

- Clearly define a DRI (Directly Responsible Individual) who has the final say on subjective matters.

Time-Boxing

- Limit the discussion for trivial items (e.g., "We have 5 minutes to pick a name, or we go with Option A").

Filter Feedback

- Differentiate between "Critical/Blocking" feedback and "Nitpicks" (subjective preferences).

What is JSON:API?

A Shared Convention

- A specification for building APIs in JSON to "stop bikeshedding" (wasteful internal debates) over API structure.

Efficiency First

- Designed to minimize the number of HTTP requests and the amount of data transmitted between client and server.

Relationship-Centric

- It acts as a "wire protocol" for fetching and updating a graph of data over HTTP.

Core Document Structure

Top-Level Members

- `data:` The primary data (resource object or collection).
- `errors:` An array of error objects (if any).
- `meta:` Non-standard meta-information.
- `included:` Related resource objects (for compound documents).
- `links:` Links related to the primary data (e.g., `self, next`).

Resource Objects

- Must contain `id` and `type` to identify the resource.

Key Features & Examples

Compound Documents (include)

- Concept: Fetch primary data and related resources in a single request.
- Example: `GET /articles/1?include=author,comments`
- Benefit: Eliminates the "N+1 request problem.”

Sparse Fieldsets (fields)

- Concept: Request only the specific attributes you need.
- Example: `GET /articles?fields[articles]=title,body`
- Benefit: Reduces payload size significantly.

Sorting, Pagination, and Filtering

- Sorting: `GET /articles?sort=-created,title`
- Pagination: `GET /articles?page[number]=2&page[size]=10`
- Filtering: `GET /articles?filter[post_type]=blog`

Crucial Differences (vs. Standard JSON)

Strict Structure

- Unlike a "wild west" JSON API, every response follows a predictable schema.

Resource Linkage

- Relationships are explicitly defined under a relationships key, providing URLs to fetch related data.

Standardized Errors

- Error responses provide specific code, title, and detail, making it easier for client-side error handling.

Best Practices (for Implementation)

Consistent Naming

- While the spec is agnostic, the community recommends **camelCase** for member names.

Use HTTP Methods Properly

- `GET` for fetching.
- `POST` for creating.
- `PATCH` for updates (JSON:API prefers `PATCH` over `PUT` for partial updates).
- `DELETE` for removing.

Leverage Caching

- Use the standardized structure to implement efficient client-side caching.

Don't Reinvent the Wheel

- Use existing libraries (available for Ruby, Python, PHP, JS, etc.) to handle the serialization/deserialization.

Anti-Bikeshedding

- Stick to the spec even if it feels verbose; the predictability for clients is worth the extra bytes.

RESTful API vs. JSON:API

Feature	RESTful API (General)	JSON:API (Specification)
Identity	Architecture Style	Implementation Spec
Structure	Custom / Ad-hoc	Standardized / Predictable
Payload size	Fixed per endpoint	Optimized via Sparse Fieldsets
Relationships	Manual / Custom	Built-in via *included* & *link*
Integration	Requires custom documentation	Works with many off-the-shelf tools

What is RPC (Remote Procedure Call)?

Definition

- A protocol that allows a program to execute a procedure (a function) on a remote server as if it were a local function call.

Goal

- To hide the complexity of network communication from the developer.

How it Works

- **Client Stub:** A local proxy that packs the parameters (Marshalling).
- **Network:** Sends the request to the server.
- **Server Skeleton:** Unpacks the parameters (Unmarshalling) and calls the actual function.

Usage Pattern

- It focuses on "verbs" or commands (e.g., `getUser(), calculateTax(), deletePost()`).

RESTful API vs. RPC

Feature	RESTful API (General)	RPC
Philosophy	Resource-based (Nouns/Objects)	Action-based (Verbs/Functions)
Coupling	Loose: Client interacts with URIs	Tight: Client must know the function name
HTTP Verbs	Uses *GET, POST, PUT, DELETE*, etc.	Usually uses *POST* for everything
Payload	Usually Binary, XML, or JSON	Usually JSON or XML
Example URL	*DELETE /users/123*	*POST /deleteUser*

What is gRPC?

Definition

- A high-performance, open-source universal Remote Procedure Call (RPC) framework.

Origins

- Developed by Google (originally called Project Stubby).

Core Technology

- Uses **HTTP/2** for transport and **Protocol Buffers (Protobuf)** as the Interface Definition Language (IDL).

Language Agnostic

- Supports 10+ languages (Go, Java, Python, C#, Node.js, etc.) with automatic client/server code generation.

Key Components

Protocol Buffers (Protobuf)

- A language-neutral, platform-neutral mechanism for serializing structured data. It is binary, making it smaller and faster than JSON/XML.

HTTP/2 Transport

- Enables features like
  - **Multiplexing:** Sending multiple requests over a single connection.
  - **Header Compression (HPACK):** Reduces overhead.
  - **Bi-directional Streaming:** Real-time data flow in both directions.

The Four Lifecycle Patterns

Unary RPC

- Simple Request-Response (Client sends one request, gets one response).

Server Streaming

- Client sends one request; server sends back a stream of multiple messages.

Client Streaming

- Client sends a stream of messages; server responds with a single message.

Bi-directional Streaming

- Both sides send a sequence of messages using a read-write stream.

Why Choose gRPC?

Performance

- Up to $10\times$ faster than REST with JSON in high-load scenarios.

Strict Typing

- Errors are caught at compile-time rather than runtime.

Streaming Support

- Native support for real-time updates without WebSockets hacks.

Deadlines/Timeouts

- Built-in support for propagating timeouts across services.

Best Practices

Use Proto3

- Always use the latest version of Protocol Buffers (proto3) for better compatibility.

Semantic Versioning

- Never change field numbers in `.proto` files to maintain backward compatibility.

Load Balancing

- Use **Client-side load balancing** or **Lookaside load balancing** (gRPC connections are long-lived, so traditional L4 load balancers may cause uneven traffic).

Deadlines

- Always set a `Deadline` for every RPC to prevent hanging requests from cascading across microservices.

Security

TLS/SSL

- Always use TLS for production traffic (gRPC-protected connections).

Interceptors

- Use "Interceptors" (similar to Middleware) for authentication, logging, and monitoring.

Real-world Usage Examples

Microservices

- Inter-service communication inside a data center (e.g., Netflix, Square).

Polyglot Systems

- When Service A (Java) needs to talk to Service B (Python) with high performance.

Real-time Dashboards

- Pushing live stock prices or sensor data to a monitoring app.

Mobile Clients

- Using gRPC-Web or native gRPC for efficient mobile-to-backend communication.

gRPC vs. Standard RPC

- **gRPC** is not a replacement for RPC; it is a **modern, high-performance implementation** of the RPC framework created by Google.
- **Transport Layer**
  - **Traditional RPC -** Often uses TCP or HTTP/1.1.
  - **gRPC -** Built on HTTP/2, allowing for multiplexing, header compression, and persistent connections.
- **Serialization Mechanism**
  - **Traditional RPC -** Uses XML (SOAP) or JSON, which are text-heavy and slower to parse.
  - **gRPC -** Uses Protocol Buffers (Protobuf), a binary format that is much smaller and faster to serialize/deserialize.
- **Streaming Capabilities**
  - **Traditional RPC-** Mostly limited to unary (request-response).
  - **gRPC -** Native support for bi-directional streaming (Server to Client, Client to Server, or both simultaneously).
- **Code Generation**
  - **Traditional RPC -** Often requires manual boilerplate code for different languages.
  - **gRPC -** Automatically generates client/server stubs for over 10 languages using the .proto file.

Homework

- Build a standardized RESTful API that follows the JSON:API specification.
  - Create a Node.js Express server with a GET /api/books/:id endpoint.
  - Define a Book resource with the following fields: title, isbn, and author.
  - Implement an action-oriented API using JSON-RPC.
- You are developing a controller for a Smart Home. Unlike REST (which manages resources), RPC is better suited for "Actions" (commands). You need to tell the house to perform a specific task.
  - Create a JSON-RPC 2.0 server using the jayson library.
  - Implement a method (verb) called toggleSmartLight.
  - The method should accept two parameters: roomName (string) and brightness (integer).
  - The server should return a success message: "Light in [roomName] set to [brightness]%".
- Build a high-performance streaming service using gRPC and Protocol Buffers.
  - Create a .proto file with a service named CryptoService.
  - Define a rpc StreamPrices (Empty) returns (stream PriceUpdate).
  - Implement a Node.js gRPC server that "pushes" a random price every 500ms.
  - Implement a listener that prints "Current BTC Price: $[value]" to the console as the data arrives.